

BAB 26: SATUAN CSS

Bagian III: Fondasi CSS

Dari Struktur ke Layar: Desain Web bagi Noncoder

Lukman Zaman · Institut Sains dan Teknologi Terpadu Surabaya

Target Pembelajaran

1. Memilih satuan CSS yang tepat sesuai maksud desain (*Design Intent*).
2. Membedakan satuan absolut (`px`) dan satuan relatif tipografis (`rem` , `em` , `ch`).
3. Menguasai satuan viewport modern (`vw` , `vh` , `dvh` , `svh`) untuk tata letak layar penuh.
4. Menerapkan fungsi matematika CSS modern (`calc()` , `min()` , `max()` , dan formula cair `clamp()`).

Pertanyaan Pembuka

"Mengapa mengunci ukuran huruf teks dengan satuan `px` dianggap sebagai pelanggaran etika aksesibilitas bagi jutaan lansia dan pengguna tunanetra?"

Pengguna Mengatur Ukuran Font Peramban ke 'Very Large' (24px):

Situs A (Memakai font-size: 16px kaku):
✗ Teks MENGABAIKAN preferensi pengguna! Teks tetap kerdil 16px.
Situs B (Memakai font-size: 1rem relatif):
✓ Teks OTOMATIS membesar proporsional menjadi 24px! Pengguna nyaman membaca.

- Di web, kita tidak mengontrol ketajaman mata pembaca kita. Satuan CSS yang kita pilih menentukan apakah desain kita bersahabat atau diskriminatif.

 Untuk Mata Desainer: Matriks Pemilihan Satuan

Satuan	Relatif Terhadap	Penggunaan Terbaik di Desain UI
px	Piksel fisik layar	Garis border halus (1px), bayangan mikro, radius kecil
rem	Ukuran font elemen akar (<html>)	Ukuran font seluruh teks , margin/padding tata letak global
em	Ukuran font elemen induk / dirinya	Padding tombol & ikon yang harus membesar otomatis saat font membesar
ch	Lebar angka nol (0) dari font aktif	Panjang baris bacaan (<i>Measure</i>) pada paragraf artikel
%	Dimensi elemen wadah induk	Lebar kolom grid, tata letak fluid multi-kolom
vw / vh	Lebar & tinggi jendela peramban	Seksi hero layar penuh (<i>Full-screen Hero Section</i>)

rem vs em: Jangan Tertukar!

rem (Root EM)

- Selalu mengacu pada font `<html>` (standar bawaan browser: `1rem = 16px`).
- **Stabil & Terprediksi:** Di mana pun diletakkan, `1.5rem` selalu sama ukurannya.

```
/* Ukuran tipografi yang stabil */
h1 { font-size: 2rem; } /* 32px */
p { font-size: 1rem; } /* 16px */
```

em (Element EM)

- Mengacu pada `font-size` elemen itu sendiri atau induk terdekatnya.
- **Dinamis & Proporsional:** Sangat cocok untuk komponen mandiri!

```
/* Tombol yang adaptif secara proporsional */
.tombol {
  font-size: 1.25rem;
  /* Padding ikut membesar otomatis jika font tombol diubah */
  padding: 0.5em 1.0em;
}
```

⚠️ Bahaya Efek Bergulung (*Compounding em Bug*)

Jika Anda menggunakan satuan `em` untuk ukuran font pada elemen yang bersarang bertingkat, ukurannya akan berlipat ganda tak terkontrol:

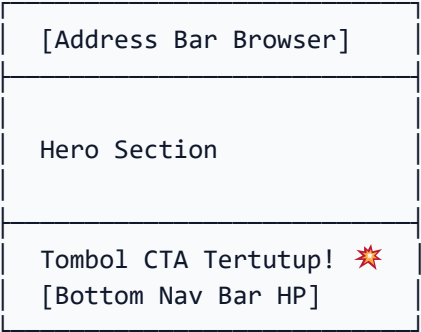
```
<ul> <!-- font-size: 1.2em; (16px × 1.2 = 19.2px) -->
  <li>Menu 1
    <ul> <!-- font-size: 1.2em; (19.2px × 1.2 = 23.04px) -->
      <li>Submenu 1
        <ul> <!-- font-size: 1.2em; (23.04px × 1.2 = 27.6px) 🌟 -->
          <li>Membengkak raksasa!</li>
        </ul>
      </li>
    </ul>
  </li>
</ul>
```

🔔 **Aturan Mutlak DKV:** Untuk `font-size`, selalu gunakan `rem`. Gunakan `em` hanya untuk padding/margin lokal komponen!

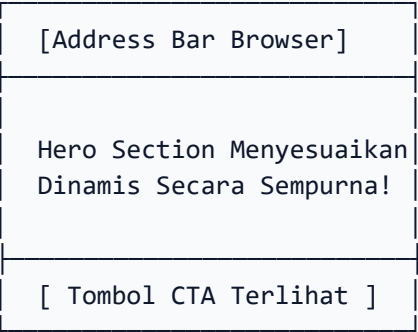
Satuan Viewport: Menaklukkan Layar Ponsel

Pernahkah Anda memberi `height: 100vh;` pada hero banner, lalu di ponsel bagian bawahnya tertutup oleh bilah alamat (*address bar*) Safari/Chrome?

Masalah 100vh Klasik:



Solusi Modern CSS: 100dvh



✔ Gunakan `100dvh` (Dynamic Viewport Height): Otomatis menyusut dan memanjang saat bilah browser muncul/sembunyi!



Fungsi Matematika di CSS

1. `calc()` (Perhitungan Campuran)

Menggabungkan satuan berbeda (misal: persen dikurangi piksel tetap):

```
.kolom-utama {  
  /* Lebar 100% layar dikurangi bilah sisi 280px */  
  width: calc(100% - 280px);  
}
```

2. `min()` dan `max()` (Batas Cerdas)

```
.kotak-dialog {  
  /* Pilih nilai terkecil: di desktop 600px, di HP 90% layar */  
  width: min(600px, 90%);  
}  
  
.teks-peringatan {  
  /* Jangan pernah lebih kecil dari 14px */  
  font-size: max(14px, 1vw);  
}
```


✨ Tipografi Cair Modern dengan `clamp()`

Fungsi `clamp()` memungkinkan ukuran font membesar dan mengecil secara mulus (*fluid*) mengikuti lebar layar tanpa perlu satu pun `@media` query!

`clamp(Minimum, Preferensi Cair, Maksimum)`

```
/* Judul Artikel Cair Responsif */
h1.tajuk-cair {
  /*
    - Di layar HP kecil: Tidak akan pernah kurang dari 2rem (32px)
    - Di tablet/laptop: Tumbuh mulus seiring lebar layar (4vw + 1rem)
    - Di monitor lebar 4K: Berhenti membesar di 4rem (64px)
  */
  font-size: clamp(2rem, 4vw + 1rem, 4rem);
  line-height: 1.15;
}
```

✨ **Satu Baris Ajaib:** Menggantikan 4 baris kode media query yang kaku dan terputus-putus!

⚠ Jebakan Umum Pemula

1. Mengubah Base Font di `html` Jadi Persen Aneh

Mengubah `html { font-size: 62.5%; }` demi trik jadul `1rem = 10px`.

Risiko: Merusak kalkulasi komponen dari pustaka pihak ketiga dan membingungkan desainer lain. Biarkan root default `16px = 1rem`.

2. Menulis `calc(100% - 20px)` Tanpa Spasi

```
/* Rusak dan Diabaikan Browser: */  
width: calc(100%-20px);  
/* Wajib ada spasi di sekitar operator matematika: */  
width: calc(100% - 20px);
```

3. Menggunakan `vw` Murni untuk Font

Memberi `font-size: 5vw;` pada paragraf body. Di layar HP teks menjadi kerdil tidak terbaca, di layar ultrawide menjadi raksasa mengerikan.

Solusi: Selalu kunci dengan `clamp()` !

Latihan Studio: Mengonversi Satuan Statis ke Fluid

Langkah 1: BACA

Periksa berkas lama `B26-L01.css` yang serba kaku dalam piksel:

```
/* Kode Lama Berbasis Piksel Statis */
body { font-size: 16px; }
.wadah { width: 960px; margin: 0 auto; }
.banner-hero { height: 600px; }
.judul { font-size: 48px; }
.kolom-teks { width: 650px; }
```

Langkah 2: UBAH (Soal Mandiri)

1. Ubah `body` menjadi `font-size: 1rem;` dan gunakan `rem` untuk margin tipografi.
2. Ubah `.wadah` menjadi `width: min(960px, 100% - 3rem);`.
3. Ubah `.banner-hero` menjadi `min-height: 80dvh;`.
4. Jadikan `.judul` cair dengan `font-size: clamp(2rem, 5vw, 3.5rem);`.
5. Ubah `.kolom-teks` agar nyaman dibaca dengan `max-width: 65ch;`.

Latihan Studio: Merakit Komponen Tombol Skalabel

Langkah 3: PERLUAS

Buat sistem tombol yang dapat berubah skala secara proporsional hanya dengan mengganti ukuran font:

```
.btn {  
  font-size: 1rem;  
  padding: 0.6em 1.2em; /* Relatif terhadap font tombol! */  
  border-radius: 0.3em;  
}  
.btn--besar {  
  font-size: 1.35rem; /* Padding otomatis ikut membesar! */  
}
```

Langkah 4: CIPTA

Rancang tata letak sampul portofolio interaktif:

1. Pasang kontainer layar penuh yang adaptif pada ponsel (`min-height: 100dvh`).
2. Terapkan tipografi cair `clamp()` pada judul proyek utama.
3. Uji dengan fitur *Responsive Mode* di browser dari ukuran 360px hingga 1920px!

Ringkasan & Kosakata Kunci

Poin Inti Bab 26

1. **Aksesibilitas rem:** Selalu gunakan `rem` untuk tipografi agar preferensi font pengguna dihormati.
2. **Proporsional em:** Manfaatkan `em` untuk padding komponen mandiri yang adaptif terhadap ukuran teksnya.
3. **Kenyamanan ch:** Batasi lebar kolom bacaan dengan satuan `ch` (karakter).
4. **Fluiditas clamp():** Bangun tipografi responsif mulus dengan formula `clamp(min, preferred, max)`.

 **Berikutnya:** Bab 27 akan menutup Bagian III dengan senjata pamungkas sistem desain: **Custom Properties (Variabel CSS)** — satu tempat untuk mengendalikan seluruh tema dan warna!

Glosarium Bab 26

- **rem:** *Root EM*, satuan relatif terhadap ukuran font elemen akar `<html>`.
- **em:** Satuan relatif terhadap ukuran font elemen itu sendiri/induknya.
- **ch:** Satuan lebar relatif terhadap glyph angka nol (`0`) dari font aktif.
- **dvh:** *Dynamic Viewport Height*, tinggi layar yang aktif menyesuaikan toolbar browser mobile.
- **clamp():** Fungsi CSS untuk mengunci nilai di antara batas minimum, nilai cair, dan batas maksimum.